



UNIVERSITY OF PERADENIYA
FACULTY OF SCIENCE
DEPARTMENT OF STATISTICS AND COMPUTER SCIENCE
END SEMESTER EXAMINATION - SEMESTER II (2017/2018)
CS 105 – Object Oriented Programming Practical (1 Credit)

Time Allowed: **THREE Hours**

Answer **ALL Questions**

1.

The *City Library* is planning to computerize their work in order to provide an efficient service. As the first step you are asked to create a system that can be used to handle the report generation of the Library, specifically to calculate and generate the reports for fining details for each user. Create a new Project named **S16xxxQ1** and implement the functionality described below.

- a. Create a method to read data from the file “*fineDetails.txt*” stored in your home (H:/) drive and store the data using a suitable data structure. Two sample rows of the data is given below.

Note : Actual text file (fineDetails.txt) contains comma separated data fields.

User Name	Book Name	No of Days (Late)
Nimal Karunarathna	The Bourne Identity	5
Shan Neththasinghe	The List	15

- b. Create another method that can be used to calculate fines for each user and write them to a file named *fineReport.txt*. The Calculation details are as follows.

Fine period - After return date	Fine Amount
First week (from day 1 until day 7)	Rs. 20 per day
Second week (from day 8 until day 14)	Rs. 100 per day
After second week	Rs. 150 per day

Eg: Fine for Nimal Karunarathna => (5 * 20) = **100/=**

Fine for Shan Neththasinghe => (7 * 20 + 7 * 100 + 1 * 150) = **990/=**

The data should be written as follows,

Nimal Karunarathna, 100

Shan Neththasinghe, 990

[P. T. O]

2.

As the second step you are asked to create a system that can be used to manage all the books and users working in the library. Create a new project named **S16xxxQ2**.

There are **three** main types of reading material that should be managed by the system (**books, journals, and magazines**),

- All the books should store their name, author's name (assume that each book is written by only one author), name of the publisher, total number of copies in the library, currently available copies (*not borrowed*) out of the total and whether the book has an age restriction level or not. The following functionality should be implemented for all the users as well,
 - Add copies to the total number of copies.
 - Remove copies from the total number of copies
 - Print all details of a book.
- Journals and Magazines are also treated as books with **additional properties and behaviors** as described below,
 - Journals should contain a volume number and this number should be also included when printing the details of the journals.
 - Magazines should contain an issue number as well as the printed year which should be also included when printing the details of Magazines.

There are **three** types of user roles that has been found to use the proposed system (**member, librarian, and administrator**),

- The system should store a unique ID, full name , email address, age regardless of the role of the user. It should also include a maximum book limit that can be borrowed at a given time, a list containing the information about the books that a particular user has borrowed (*book name, allowed lending duration in days*) and their access level. The system should also be able to print all the details of any user.
 - Apart from the above mentioned functionality, the following restrictions should be added for each role,
 - A Library Member (access level = 3) has a book limit of two books
 - A Librarian (access level = 2) has a maximum book limit of three books.
 - An Administrator (access level = 1) has a maximum limit of five books.
- a. Implement the above mentioned attributes and behaviors in each of the created Classes (create getter and setter methods as well as custom constructors if necessary).
- b. Create a suitable data structure in the class **BookStore** that can be used to store Books. This data structure should have only *one instance for all the objects* created using this class and should be able to *access only within the class itself*.

Question 2.b Continued to next page.....

Question 2.b Continued

- Create methods for adding, removing, searching & retrieving books from this data structure (Duplicates should not be allowed when adding).
 - Create a method to print details of all the books stored in this data structure.
- c. Create a suitable data structure in the class **UserStore** that can be used to store all the Users. This data structure should have only *one instance for all the objects* created using this class and should be able to *access only within the class itself*.
- Create methods for adding removing and retrieving Users from this data structure. (Duplicate users should not be allowed)
 - Create a method to print details of all the users stored in this data structure.
- d. Create a method in the class **User** that can be used to generate the unique ID for each user . The ID should be Started with the letter 'U' at the beginning and should have additional 4 digits which should contain the next number in the user-list which you have created in *Question 2.c*.

New uses's ID = "**U0010**"

Hint : This method should be used to set each users ID automatically at the user creation time.

Eg: Total number of current users in the list = 9

New user's number = (9+1) = 10

- e. Create following methods for **Member**,
- Method that can be used to borrow books. When borrowing books, the number of borrowed books of the user should not exceed the book limit after burrowing the new book and the number of copies currently in the library of that particular book should be updated accordingly.(The default lending time limit is 14 days)
 - Method to return the borrowed books. When returning book, the user's currently borrowed books list and the number of copies in currently in the library should be updated accordingly.
- f. Create following methods for **Librarian**.
- Method to add copies to a currently existing book in the book-list created in *Question 2.b*.
 - Method to add new books to the Library. (When trying to add a book which is already present in the library, it should be added as a *copy* to the existing book).
 - Method to remove copies of a book from the book-list. If the copy count is equal to total number of copies of the book. The whole book should be removed from the list.

Question 2.f Continued to next page

[P. T. O]

Question 2.f Continued

- Method to increase the borrowed time of a particular book borrowed by a particular user by 7 days.

g. Create the following methods for the Administrator.

- Method to add a user according to the user level.

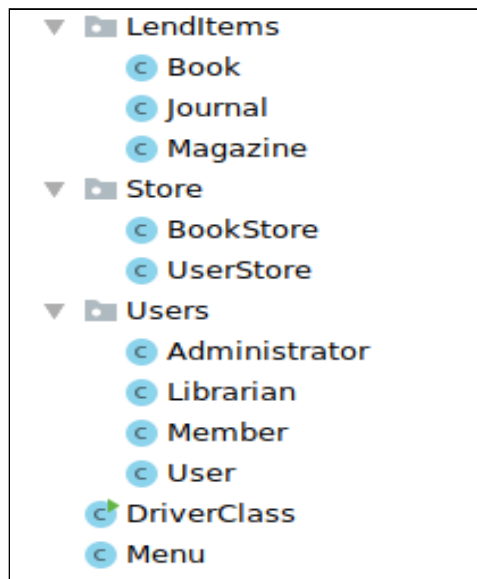
Eg: If the user level is stated as 2 a new Librarian should be added to the user-list.

- Method to remove a user.
- Method to update a user according to the user level.

Eg: if the user level is changed from 3 to 2 the current user (who is a Library Member) should be removed from the user-list and should be added as a new Librarian with other properties like the borrowed booklist remaining unchanged.

- h. The file `Menu.java` contains a user-interface created for the above system. Copy the file into your project and implement the functionality using the methods created in the above parts.**

Sample project structure of the project is given below.



[End of Paper]